

Contents

Feature Implementation Runbook	1
1. Firebase Auth (signup/login, email verification, password reset, logout, live deactivation)	1
2. Firebase App Check (reCAPTCHA v3)	3
3. Firestore Security Rules — role-based access control	5
4. Server-side rate-limited “claim” API pattern	6
5. Push Notifications — OneSignal (Web SDK + Median native bridge)	8
6. Server-side notification dispatch + status-transition routing	11
7. Scheduled cron job (mark-overdue)	12
8. Image upload/compression pipeline	14
9. PDF generation (jsPDF)	16
10. Bounded/paginated Firestore queries + composite indexes	17
11. API routing on Vercel (SPA + serverless functions coexisting)	18
12. Median native app wrapper (Android/iOS shell + native OneSignal bridge)	19

Feature Implementation Runbook

Reproducible setup procedures for each major feature in this codebase. Written for replicating these in a new project — not an architecture overview.

Sections are ordered by dependency: Auth → App Check → Firestore Rules → everything that needs a logged-in/verified user.

1. Firebase Auth (signup/login, email verification, password reset, logout, live deactivation)

1. What it does Email/password auth via Firebase Auth, gated behind email verification, with a role stored in a separate `users/{uid}` Firestore doc, client-side login-attempt logout, and a live listener that force-signs-out a user the instant their account is deactivated or deleted — not just on next login.

2. Outside-the-codebase setup 1. Firebase Console → Authentication → Sign-in method → enable **Email/Password**. 2. Firebase Console → Authentication → Templates → Email address verification → confirm/customize the template (the app calls `sendEmailVerification` with a custom `url` action setting, so no dynamic link domain config is required beyond the default). 3. Firebase Console → Authentication → Settings → Authorized domains → add your production domain (and any Median/native wrapper domain if different). 4. No Blaze plan required for Auth itself, but Blaze is required later for App Check enforcement + Admin SDK server routes on Vercel serverless functions with network egress (see App Check section).

3. Environment variables required

Variable	Client/Server	Where to obtain	Declared/read in
VITE_FIREBASE_API_KEY	Client	Firebase Console → Project Settings → General → Web app config	src/firebase.js
VITE_FIREBASE_AUTH_DOMAIN	Client	same	src/firebase.js
VITE_FIREBASE_PROJECT_ID	Client	same	src/firebase.js

Variable	Client/Server	Where to obtain	Declared/read in
VITE_FIREBASE_STORAGE_BUCKET	Client	same	src/firebase.js
VITE_FIREBASE_MESSAGING_SENDER_ID	Client	same	src/firebase.js
VITE_FIREBASE_APP_ID	Client	same	src/firebase.js

4. Files involved, in order

File	Purpose	Essential code
src/firebase.js	Init Auth client	export const auth = getAuth(app);
signUp/signUp.jsx	Signup/login UI + logic	See order-of-operations below
firestore.rules (match /users/{userId})	Enforces profile doc shape on create, restricts self-updates	see Firestore Rules section
src/protectedRoute.jsx (or wherever your route guard lives)	Route guard + live deactivation listener	see below

Live deactivation listener (ProtectedRoute), the structurally essential part:

```
const unsubscribeUserDoc = onSnapshot(
  doc(db, "users", firebaseUser.uid),
  (userSnap) => {
    if (!userSnap.exists() || userSnap.data().deactivated === true) {
      auth.signOut();
      localStorage.removeItem("user");
      setUser(null);
      return;
    }
    setUser(userSnap.data());
    localStorage.setItem("user", JSON.stringify({ data: userSnap.data(), timestamp: Date.now() }));
  },
  () => { auth.signOut(); }
);
```

Attach/detach this inside onAuthStateChanged, dropping the previous listener before attaching a new one on every auth-state change.

Client-side lockout (in the login form), essential shape:

```
const MAX_ATTEMPTS = 5;
const LOCKOUT_MS = 30_000;
// on failure: increment attempts in localStorage; at MAX_ATTEMPTS, store
// { attempts, lockoutUntil: Date.now() + LOCKOUT_MS } and disable the form
// until lockoutUntil passes. This is a UX speed bump only – it is NOT a
// real rate limit (see Firebase Console → Authentication → Settings for
// server-side brute-force protection, which is on by default).
```

5. Order of operations

1. createUserWithEmailAndPassword — creates the Auth account. Nothing else has happened yet, so a failure here needs no cleanup.
2. Any server-side claim step that depends on this user existing (e.g. registration ID claim — see Section 4) — must run **after** step 1, because rolling back on failure means calling deleteUser(createdUser) on the Auth object from step 1.
3. Write the users/{uid} profile doc — must run after step 2,

because the profile doc's role field usually comes from the claim response. 4. `sendEmailVerification(createdUser, { url: <return-url> })`. 5. `signOut(auth)` immediately after signup — do not leave the user signed in with an unverified email. 6. On login: `signInWithEmailAndPassword` → check `user.emailVerified` → if false, sign out again and show a “resend verification” option → if true, `getDoc(users/{uid})` → check deactivated → only then persist to `localStorage` and navigate.

6. Common pitfalls encountered in this project - `currentUser` read from `localStorage` can be undefined on first render before it resolves — initializing dependent state (e.g. a form field) directly from it causes an uncontrolled→controlled input warning. Always fall back to `"/null` explicitly. - Firebase's `emailVerified` flag is only refreshed on the User object after a fresh `signInWithEmailAndPassword` or `reload()` — a stale cached user object in `localStorage` can say verified when it isn't. - Deactivating a user mid-session does nothing unless you have a **live** Firestore listener on their own user doc (a one-time `getDoc` on protected-route mount is not enough — see the listener above). - `localStorage` is a cache, never a source of truth — every read of role/deactivated status for access-control decisions must be re-verified against Firestore, not trusted from cache alone.

7. Minimal reproduction checklist - [] Enable Email/Password sign-in in Firebase Console. - [] Add authorized domains for every environment (localhost is included by default). - [] `createUserWithEmailAndPassword` → write profile doc → `sendEmailVerification` → `signOut` → show “check your email.” - [] Gate login on `user.emailVerified`. - [] Store { data, timestamp } in `localStorage` only after verifying `deactivated !== true`. - [] Attach a live `onSnapshot` listener on `users/{uid}` inside your route guard; sign out on `deactivated === true` or doc deleted. - [] Implement lockout as client-side-only UX; do not treat it as security.

2. Firebase App Check (reCAPTCHA v3)

1. What it does Attests that requests to protected server endpoints (here, the registration-ID claim API) come from your real web app, not a script — required header on top of the Auth ID token, not a replacement for it.

2. Outside-the-codebase setup 1. Firebase Console → Build → App Check → Apps → select your web app → **Register**. 2. Choose **reCAPTCHA v3** (or Enterprise) as the provider → this auto-creates a reCAPTCHA site key, or paste one from the [Google reCAPTCHA admin console](#) if you created it there first. 3. Copy the **site key** (public, goes in client env) — a separate **secret key** is generated but not needed client-side; the Admin SDK verifies tokens server-side using the Firebase project's own credentials, not the reCAPTCHA secret directly. 4. Firebase Console → App Check → APIs tab → for any Firebase product you want enforcement on (Firestore, if you go that route) toggle **Enforce**. This project enforces App Check at the custom API route level instead (see Section 4), not via Firestore's built-in App Check enforcement toggle — so this step is optional here. 5. Local development: App Check debug tokens are required because reCAPTCHA v3 does not validate on localhost reliably. - Firebase Console → App Check → Apps → your web app → : → **Manage debug tokens** → **Add debug token** (or let the app auto-generate one — see pitfall below — and paste that value in).

3. Environment variables required

Variable	Client/Server	Where to obtain	Declared/read in
<code>VITE_RECAPTCHA_V3_SITEKEY</code>	Client	Firebase Console → App Check → Apps → your app → site key	<code>src/firebase.js</code>

No server-side App Check env var is needed — `firebase-admin's getAppCheck().verifyToken()` uses the same service-account credentials already configured for the Admin SDK (see Section 4).

4. Files involved, in order

File	Purpose	Essential code
<code>src/firebase.js</code>	Initialize App Check on the client	see below
<code>pages/api/claim-registration-id.js</code> (or any protected route)	Verify the App Check token server-side	see below

```
// src/firebase.js – must run before any Firestore/Auth network call
if (import.meta.env.DEV) {
  self.FIREBASE_APPCHECK_DEBUG_TOKEN = true; // logs a debug token to console on first run
}
export const appCheck = initializeAppCheck(app, {
  provider: new RecaptchaV3Provider(import.meta.env.VITE_RECAPTCHA_V3_SITE_KEY),
  isTokenAutoRefreshEnabled: true,
});

// server route – verify before doing anything else
import { getAppCheck } from "firebase-admin/app-check";
const appCheckToken = req.headers["x-firebase-appcheck"];
if (!appCheckToken) return res.status(401).json({ error: "Missing App Check token" });
await getAppCheck().verifyToken(appCheckToken); // throws on invalid
```

Client call site attaches the token as a header:

```
const { token: appCheckToken } = await getAppCheckToken(appCheck, false);
fetch("/api/your-route", { headers: { "X-Firebase-AppCheck": appCheckToken, ... } });
```

5. Order of operations 1. `initializeAppCheck` must run immediately after `initializeApp`, before any other Firebase SDK call that might fire (Auth, Firestore) — otherwise those early calls go out unattested. 2. Only call `initializeAppCheck` once per app instance — calling it twice throws. 3. Server-side verification must happen before Auth ID token verification is *trusted* for anything sensitive, though the order of the two checks in code doesn't strictly matter as long as both gates pass before the protected logic runs.

6. Common pitfalls encountered in this project - reCAPTCHA v3 silently fails/misvalidates on localhost — you'll see 401s from your own protected routes in local dev with no useful browser-side error unless you set up a debug token. - The debug token is logged to the browser console the first time `initializeAppCheck` runs with `FIREBASE_APPCHECK_DEBUG_TOKEN = true` set — easy to miss if you're not watching the console on first load. - Forgetting to gate the debug-token line behind `import.meta.env.DEV` ships a permanent App Check bypass to production.

7. Minimal reproduction checklist - [] Register web app with reCAPTCHA v3 in Firebase Console → App Check. - [] Add `VITE_RECAPTCHA_V3_SITE_KEY` to env. - [] `initializeAppCheck` immediately after `initializeApp`, gated debug-token line behind DEV. - [] Generate/copy a debug token to Firebase Console for local dev. - [] On any protected API route: read `x-firebase-appcheck` header → `getAppCheck().verifyToken()` → 401 on missing/invalid. - [] Client attaches `X-Firebase-AppCheck` header via `getToken(appCheck, false)` on every protected fetch.

3. Firestore Security Rules — role-based access control

1. What it does Enforces a per-role, per-status state machine directly in Firestore rules: who can read which documents, and — critically — which exact fields a given role/status transition is allowed to touch, rejecting any write that touches an unlisted field.

2. Outside-the-codebase setup 1. Firebase Console → Firestore Database → create database (Native mode) if not already created — choose a region. 2. No Blaze plan required purely for rules, but composite indexes at scale (Section 10) and any Cloud Functions do require Blaze. 3. Deploy rules: `firebase deploy --only firestore:rules` (requires `firebase-tools` installed and `firebase login + firebase use <project-id>` configured, with a `firebase.json` pointing `firestore.rules` at your rules file).

3. Environment variables required None — rules are deployed via the Firebase CLI, not read from app env vars.

4. Files involved, in order

File	Purpose
<code>firebase.json</code>	Points the CLI at <code>firestore.rules</code> and <code>firestore.indexes.json</code>
<code>firestore.rules</code>	The actual rule set
<code>firestore.indexes.json</code>	Composite indexes (see Section 10)

Essential structural pattern — helper functions + per-collection allow blocks scoped by exact field diffs:

```
function getUserRole() {
  return get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role;
}

match /reports/{reportId} {
  allow update: if isVerified() && (
    (isWorker() &&
      resource.data.assignedTo == getUserID() &&
      resource.data.status == 'assigned' &&
      request.resource.data.status == 'accepted' &&
      request.resource.data.diff(resource.data).affectedKeys().hasOnly(['status', 'dateAccepted'])
    // ... one such clause per (role, fromStatus, toStatus) transition
  );
}
```

5. Order of operations / dependency notes - Rules must be deployed **before** any client code that depends on a new transition — a client write that isn't yet covered by a rule clause fails with permission-denied, which is indistinguishable in the UI from a real authorization failure unless you log the raw Firestore error. - `get()` calls inside rules (e.g. reading the caller's own `users/{uid}` doc to check role) count against your Firestore read quota and add latency to every rule evaluation that uses them — minimize repeated `get()`s by wrapping in a helper function (as above) so you can see all read cost in one place, though each rule evaluation still re-executes it. - Any collection not explicitly matched falls through to the final match `/document=*` { allow read, write: if false; } catch-all — always keep this as the last rule.

6. Common pitfalls encountered in this project - `serverTimestamp()` **cannot** be used as an element inside an array passed to `arrayUnion()` — Firestore rejects the write. Any timestamp stored inside a `notes/history` array element must be a plain client-side

new Date() instead, which introduces client-clock trust for that one field only. - request.resource.data.diff(resource.data).affectedKeys().hasOnly([...]) is strict and exact — forgetting to list a field the client’s updateDoc call also touches (even a field left unchanged, if it’s included in the payload object) causes a false rejection. - A single collection can accumulate a long OR-chain of allow update clauses, one per (role, from-status, to-status) triple. This becomes unreadable fast; keeping a comment mapping each clause to the corresponding client-side “canUserX” helper function (in the app’s utils file) prevents the two from drifting out of sync. - Rules can’t validate against data that requires cross-document business logic beyond a single get() — e.g. “is this the 6th failed attempt in 10 minutes” (rate limiting) has to live in a server-side Admin SDK route instead (Section 4), not in rules.

7. Minimal reproduction checklist - [] firebase.json references firestore.rules and firestore.indexes.json. - [] Write helper functions (isAuthenticated, getUserData, getUserRole, per-role booleans) once at the top of firestore.rules. - [] For every mutable collection, write one allow update clause per legal (role, fromStatus, toStatus) transition, each scoped with .diff().affectedKeys().hasOnly([...]). - [] Never use serverTimestamp() inside arrayUnion() array elements — use new Date() there instead. - [] End the rules file with a deny-all catch-all match. - [] firebase deploy --only firestore:rules before shipping any client code that depends on a new rule clause.

4. Server-side rate-limited “claim” API pattern

(Depends on: Section 1 Auth, Section 2 App Check, Section 3 Firestore Rules)

1. What it does A Vercel serverless API route that lets a newly-created Auth user “claim” a short numeric registration PIN server-side (via Admin SDK, bypassing normal client Firestore rules for the used flag), protected by Auth + App Check + a dual per-uid/per-IP rate limiter implemented as a Firestore transaction.

2. Outside-the-codebase setup 1. Firebase Console → Project Settings → Service Accounts → **Generate new private key** → downloads a JSON credential. Extract project_id, client_email, private_key from it for the Admin SDK env vars below. Store the file securely; do not commit it. 2. Vercel dashboard → your project → Settings → Environment Variables → add the Admin SDK vars (see table) for Production, Preview, and Development as needed. 3. Confirm your Firestore project is on a plan that allows the read/write volume of the rate-limiter’s transactional counter doc (claimAttempts/{key}) — negligible cost on Spark for normal traffic, but the collection should be excluded from any TTL/backup policies you don’t want it touching.

3. Environment variables required

Variable	Client/Server	Where to obtain	Declared/read in
FIREBASE_ADMIN_PROJECT_ID	Server	Service account JSON → project_id	pages/api/claim-registration-id.js
FIREBASE_ADMIN_CLIENT_EMAIL	Server	Service account JSON → client_email	same
FIREBASE_ADMIN_PRIVATE_KEY	Server	Service account JSON → private_key (keep \n escaped in the env var; code does .replace(/\n/g, "\\n"))	same

(Inferred from usage in the API route — no `.env.example` was present in the shared files; confirm the exact var names against your own Vercel project settings.)

4. Files involved, in order

File	Purpose
pages/api/claim-registration-id.js	The protected claim endpoint
firestore.rules (match /registrationIDs/{registrationId})	Client-facing read + narrowly-scoped self-release rule (see below)
signup/signup.js	Client call site, in the middle of the signup flow

Essential server skeleton:

```
if (!getApps().length) {
  initializeApp({ credential: cert({
    projectId: process.env.FIREBASE_ADMIN_PROJECT_ID,
    clientEmail: process.env.FIREBASE_ADMIN_CLIENT_EMAIL,
    privateKey: process.env.FIREBASE_ADMIN_PRIVATE_KEY?.replace(/\\n/g, "\n"),
  })});
}

async function checkAndBumpLimit(key) {
  const ref = db.collection("claimAttempts").doc(key);
  await db.runTransaction(async (tx) => {
    const snap = await tx.get(ref);
    const now = Date.now();
    const data = snap.exists ? snap.data() : null;
    const windowStart = data?.windowStart ?? now;
    if (!data || now - windowStart > WINDOW_MS) {
      tx.set(ref, { count: 1, windowStart: now });
      return;
    }
    if (data.count >= MAX_ATTEMPTS) throw new Error("RATE_LIMITED");
    tx.update(ref, { count: data.count + 1 });
  });
}

// in the handler, in order:
await checkAndBumpLimit(`uid:${uid}`);
await checkAndBumpLimit(`ip:${ip}`);
await db.runTransaction(async (tx) => {
  const regSnap = await tx.get(regRef);
  if (!regSnap.exists) throw new Error("REG_NOT_FOUND");
  if (regSnap.data().used === true) throw new Error("REG_ALREADY_USED");
  // ... expiry + type checks ...
  tx.update(regRef, { used: true, claimedBy: uid });
});
```

Firestore rule for the claimed doc's narrowly-scoped self-release path (used only if the profile-doc write after claiming fails, so the PIN isn't burned on a partial failure):

```
allow update: if isAuthenticated() &&
  resource.data.used == true &&
```

```
resource.data.claimedBy == request.auth.uid &&
!exists(/databases/${database}/documents/users/${request.auth.uid}) &&
request.resource.data.used == false &&
request.resource.data.diff(resource.data).affectedKeys().hasOnly(['used']);
```

The `claimAttempts` collection itself should be fully locked down — no client read/write, Admin SDK only: `match /claimAttempts/{key} { allow read, write: if false; }`.

5. Order of operations / dependency notes 1. Auth account created first (Section 1) — the claim call needs a valid ID token. 2. Client requests both an Auth ID token and an App Check token before calling the route. 3. Server verifies App Check token → verifies Auth ID token → runs rate-limit transaction(s) → runs the claim transaction. 4. If the claim succeeds but the **subsequent** profile-doc write fails client-side, the client must call the self-release rule above to set `used: false` again before also deleting the just-created Auth user — otherwise the PIN is permanently burned with no user attached to it. 5. On successful claim, clear the per-uid rate-limit counter (`clearLimit`) so a user who fat-fingered the PIN a couple of times isn't left sitting near the limit for legitimate future use. 6. **Vercel routing pitfall (folded in per request):** `/api/*` routes only resolve automatically if `pages/api/*.js` sits at the project root Vercel expects, and only under `vercel dev/deployed` builds — plain `vite dev` does **not** proxy `/api` calls, so hitting these routes returns your SPA's `index.html` (a 200, not a 404, which is what makes this pitfall so easy to miss) or a raw 404 depending on your dev setup. Use `vercel dev` locally, or add a dev-only proxy in `vite.config.js` (`server.proxy["/api"]`) pointing at a local Vercel dev process. Confirm `vercel.json`'s `rewrites` array excludes `/api/` from the SPA catch-all rewrite (see the regex `^(?!api/|.*\.[a-zA-Z0-9]+)$.*` used in this project — it explicitly excludes anything starting with `api/`).

6. Common pitfalls encountered in this project - The `FIREBASE_ADMIN_PRIVATE_KEY` env var breaks silently if the literal `\n` sequences aren't preserved through your hosting provider's env var UI — always apply `.replace(/\n/g, "\n")` when reading it, and verify by test-connecting rather than assuming it “looks right” in the dashboard. - A non-JSON response from a misconfigured/misrouted API endpoint (see the Vercel routing pitfall above) breaks `res.json()` parsing client-side with a cryptic error — always check `content-type` before parsing, and fall back to reading the response as text for logging. - Forgetting to guard `if (!getApps().length)` before `initializeApp` in a serverless function causes a “duplicate app” crash on function warm starts (the module can be re-invoked without a fresh cold start). - Short numeric PINs (this project uses 6 digits) have a much smaller keyspace than something like `nanoid()` — this makes uniqueness-checking on generation and rate-limiting on claim **required**, not optional, and the registration doc had to be made publicly get-able in rules (since the client checks the PIN before the user has an auth token) which shifts more of the security burden onto rate limiting than a long token-style ID would need.

7. Minimal reproduction checklist - [] Generate a Firebase service account key; store its 3 fields as server env vars. - [] Guard Admin SDK init with `if (!getApps().length)`. - [] Implement a Firestore-transaction-based rate limiter keyed by both `uid:` and `ip:` prefixes, in a fully-locked-down collection. - [] Verify App Check token, then Auth ID token, before any business logic. - [] Run the actual claim as its own Firestore transaction with `existence/used/expiry` checks. - [] Add a narrowly-scoped Firestore rule allowing only the exact claiming `uid` to self-release an unused-but-not-yet-profiled claim. - [] Confirm `vercel.json` rewrites exclude `/api/` from the SPA catch-all, and use `vercel dev` (not bare `vite dev`) locally. - [] Clear the uid rate-limit counter on successful claim.

5. Push Notifications — OneSignal (Web SDK + Median native bridge)

(Depends on: Section 1 Auth)

1. What it does Links each signed-in Firebase user to a OneSignal “External ID” (their Firebase UID) so server-side pushes can target them by uid regardless of platform, with two separate client code paths: OneSignal’s Web SDK (browser/iOS PWA) and Median’s native OneSignal bridge (wrapped Android/iOS app — see Section 12).

2. Outside-the-codebase setup 1. Create a OneSignal account → **New App/Website** → choose **Web Push** (and later add mobile platforms once wrapped via Median, which auto-configures native OneSignal under the hood — see Section 12). 2. OneSignal Dashboard → Settings → Keys & IDs → copy the **OneSignal App ID** and the **REST API Key**. 3. OneSignal Dashboard → Settings → Push & In-App → Web Push configuration: set your site URL, upload icons, confirm the default subscription/prompt behavior (this project calls `requestPermission()` manually instead of using OneSignal’s auto-prompt, so you can disable the auto-prompt here). 4. iOS/Android native sound + channel setup, if using a custom notification sound: upload the sound file through Median’s App Studio (Native Plugins → OneSignal) — this project references `custom_sound_1.caf` (iOS) and an Android notification channel UUID; both must be created in that dashboard, not in code, and the resulting IDs pasted into the server code (see Section 6). 5. Add the OneSignal Web SDK `<script>` tag and init block to your HTML entry point (not deferred behind a bundler import, since it must be available before React mounts).

3. Environment variables required

Variable	Client/Server	Where to obtain	Declared/read in
ONESIGNAL_APP_ID	Server	OneSignal Dashboard → Keys & IDs	<code>src/notifications/sendPushForT</code>
ONESIGNAL_REST_API_KEY	Server	OneSignal Dashboard → Keys & IDs	same
APP_BASE_URL	Server	Your deployed app’s origin (used to build absolute <code>targetUrls</code> for deep-linking)	same
(OneSignal App ID, hardcoded)	Client	Same App ID as above, pasted directly into <code>index.html</code> ’s init script (public value, safe to hardcode)	<code>index.html</code>

4. Files involved, in order

File	Purpose
<code>index.html</code>	Loads OneSignal Web SDK, calls <code>OneSignal.init({ appId })</code>
<code>src/lib/push.js</code>	<code>linkPushUser(uid)</code> — platform-detecting login/link function
<code>hooks/usePushNotification.js</code>	React hook wrapping permission state + <code>enable()</code>
<code>components/EnableNotificationsButton.jsx</code>	UI trigger for <code>enable()</code>
<code>src/notifications/sendPushForTransition.js</code>	Server-side send logic (Section 6)

```
<!-- index.html -->
<script src="https://cdn.onesignal.com/sdks/web/v16/OneSignalSDK.page.js" defer></script>
```

```

<script>
  window.OneSignalDeferred = window.OneSignalDeferred || [];
  OneSignalDeferred.push(async function (OneSignal) {
    await OneSignal.init({ appId: "<your-app-id>" });
  });
</script>

// src/lib/push.js – the platform branch is the essential structure
export async function linkPushUser(userId) {
  const uid = String(userId);
  const medianOneSignal = await waitForMedianOneSignal(); // polls window.median?.onesignal
  if (medianOneSignal) {
    medianOneSignal.login(uid);
    return true;
  }
  if (window.OneSignalDeferred) {
    return new Promise((resolve) => {
      window.OneSignalDeferred.push(async (OneSignal) => {
        await OneSignal.login(uid); // retry a few times – see pitfalls
        resolve(true);
      });
    });
  }
  return false;
}

```

5. Order of operations / dependency notes 1. `OneSignal.init()` must complete (client-side) before `OneSignal.login(uid)` is called — this project queues the login call through `OneSignalDeferred.push`, which handles the ordering for you as long as you push onto that array rather than accessing `window.OneSignal` directly. 2. `linkPushUser(uid)` should be called immediately after a successful login (see Section 1's `handleLogin`) — a user who never reaches this call never receives pushes even if they granted browser permission earlier, since the External ID link is separate from the permission grant. 3. On Median (native), the wrapper's own OneSignal bridge already handles permission prompting — the JS layer only needs to call `.login(uid)`, not request permission itself. On Web, you must call `OneSignal.Notifications.requestPermission()` yourself before `.login()` will have any subscribed device to attach to.

6. Common pitfalls encountered in this project - `window.median?.onesignal` is not guaranteed to exist immediately on page load inside the Median wrapper — a fixed-interval poll with a timeout (`waitForMedianOneSignal`) is required rather than a single synchronous check. - `OneSignal.login(uid)` can fail on the very first call in a fresh session because the SDK's internal state isn't fully ready yet — this project retries with a short backoff (5 attempts, 500ms apart) rather than surfacing the first failure as a hard error. - The Web SDK ignores data payloads on notification click for actual navigation — it uses `web_url` on the send payload instead. Native (Median) reads a custom `targetUrl` out of the data payload. Sending both is required to get correct deep-linking on every platform (see Section 6). - Detecting “unsupported” (e.g. Notification API absent, private browsing on some browsers) must be handled explicitly in the permission-state hook, or the enable button renders in a broken state with no explanation to the user.

7. Minimal reproduction checklist - [] Create a OneSignal Web Push app; get App ID + REST API key. - [] Add the SDK script + `OneSignal.init()` block to `index.html`. - [] Write a `linkPushUser(uid)` that branches on `window.median?.onesignal` vs `window.OneSignalDeferred`. - [] Call `linkPushUser` right after successful login. - [] On Web only, call `OneSignal.Notifications.requestPermission()` before `.login()`. - [] Retry

.login() a few times with backoff before giving up. - [] Store App ID + REST API key as server env vars for the send side (Section 6).

6. Server-side notification dispatch + status-transition routing

(Depends on: Section 5)

1. What it does A shared server module that maps a domain event (e.g. a report's status changing from assigned to accepted) to the correct audience (by role, or a specific user via a custom ID field) and sends via OneSignal's REST API — reused identically by both an authenticated client-triggered API route and an unauthenticated cron job.

2. Outside-the-codebase setup Same OneSignal app as Section 5 — no additional dashboard setup beyond what's already there.

3. Environment variables required Same as Section 5 (ONESIGNAL_APP_ID, ONESIGNAL_REST_API_KEY, APP_BASE_URL).

4. Files involved, in order

File	Purpose
src/notifications/resolvedTargets.js	Pure data: maps "{oldStatus}->{newStatus}" (or "*->{newStatus}") to target audiences + copy
src/notifications/sendPushForTransition.js	Resolves audiences to Firestore uids, sends via OneSignal REST API
pages/api/send-notification.js	Authenticated client-triggered entry point
src/notifications/notifyOnStatusChange.js	Client-side helper that calls the above route with an ID token
pages/api/cron/mark-overdue.js	Unauthenticated-by-client, secret-authenticated entry point (Section 7)

Essential transition-table shape:

```
export const TRANSITIONS = {
  "assigned->accepted": {
    targets: [{ audience: "role", role: "estate", route: () => "/eip" }],
    title: "Assignment Accepted",
    body: "The technician accepted the job.",
  },
  "*->overdue": {
    targets: [
      { audience: "role", role: "estate", route: () => "/eh" },
      { audience: "assignedWorker", route: () => "/wa" },
      { audience: "role", role: "admin", route: () => "/ah",
        when: (report) => ["emergency", "urgent"].includes(report?.priorityLevel) },
      { audience: "reporter", route: () => "/Pending",
        title: "Your Report Is Overdue",
        body: (report) => `... ${overdueStatusClause(report)}` },
    ],
    body: (report) => `... ${overdueStatusClause(report)}`,
  },
}
```

```
};
export function resolveTransition(oldStatus, newStatus) {
  return TRANSITIONS[`${oldStatus}->${newStatus}`] || TRANSITIONS[`${*->${newStatus}`] || null;
}
```

Essential send-side shape (chunked for OneSignal's per-call External ID cap):

```
const MAX_TARGETS = 100;
for (const target of transition.targets) {
  if (target.when && !target.when(report)) continue;
  const targetUids = await resolveTargetUids(db, target, report); // role → query users; e
  const allIds = [...targetUids];
  for (let i = 0; i < allIds.length; i += MAX_TARGETS) {
    await sendViaOneSignal({
      externalIds: allIds.slice(i, i + MAX_TARGETS),
      title, body,
      data: { reportId: report.id, status: newStatus, targetUrl }, // native deep link
      webUrl: targetUrl, // web push deep link
    });
  }
}
```

5. Order of operations / dependency notes - resolveTargetUids translates a custom app-level ID field (e.g. report.reporterId, which is *not* the Firestore doc ID / Auth uid) into actual users collection doc IDs via a where("ID", "==", customId) query — this indirection exists because the app's users documents are keyed by Firebase Auth uid, but report records store a separate human-facing custom ID. Keep this translation centralized in one function; do not duplicate the lookup elsewhere. - Per-target when(report) gates are evaluated before the Firestore uid-resolution query runs, so a gated-out target costs nothing extra. - title/body resolution precedence, most specific wins: explicit override passed by the caller → target-level title/body (which may be a function of the report) → transition-level default.

6. Common pitfalls encountered in this project - OneSignal's External ID targeting is capped per API call (100 in this project) — sending to a role with more subscribers than that silently truncates unless you chunk the ID array yourself. - Web push and native push read the destination URL from **different places** in the payload (web_url top-level field vs. data.targetUrl) — setting only one means half your platforms deep-link correctly and the other half falls back to the site root. - A “no matching users” result and a “matched users, but none have a subscribed device” result are different failure modes and were kept distinguishable in the return value (reason field) specifically so debugging silent non-delivery doesn't require re-deriving which case you're in from logs alone.

7. Minimal reproduction checklist - [] Build a {oldStatus}->{newStatus} (plus *->{newStatus} wildcard) lookup table as plain data, separate from the sending code. - [] Support per-target when(report) gating and per-target title/body overrides. - [] Centralize custom-ID → Firestore-uid resolution in one function. - [] Chunk External ID arrays to the provider's per-call cap. - [] Send both web_url and a native-readable data.targetUrl. - [] Reuse this module from both an authenticated client route and any cron/unauthenticated route rather than duplicating send logic.

7. Scheduled cron job (mark-overdue)

(Depends on: Section 6)

1. What it does A secret-authenticated HTTP endpoint, triggered by an external scheduler on

an interval, that scans for reports past their due date, marks them overdue in a single batch write, and fires notifications — with no logged-in user involved at request time.

2. Outside-the-codebase setup 1. Generate a random secret string (e.g. `openssl rand -hex 32`) — this is `CRON_SECRET`. 2. Vercel dashboard → Environment Variables → add `CRON_SECRET` (Production at minimum). 3. Set up an external scheduler pointed at your deployed endpoint on an interval: - **cron-job.org**: create a job → URL `https://<your-domain>/api/cron/mark-overdue` → Method GET or POST → Advanced → add header Authorization: Bearer `<CRON_SECRET>`. - **Vercel Cron** (if on a plan that supports it): add a `crons` array to `vercel.json` pointing at the same path; Vercel automatically sends its own internal auth, but this project's `isAuthorized` also accepts a `?secret=` query param as a fallback for schedulers that can't set custom headers — use whichever your chosen scheduler supports. 4. Confirm the endpoint's route is excluded from any SPA rewrite (see the Vercel-routing pitfall in Section 4) — cron schedulers hitting an HTML page instead of your handler is a common silent-failure mode.

3. Environment variables required

Variable	Client/Server	Where to obtain	Declared/read in
<code>CRON_SECRET</code>	Server	Self-generated random string	<code>pages/api/cron/mark-overdue.js</code>
<code>FIREBASE_ADMIN_*</code>	Server	Same as Section 4	same
<code>ONESIGNAL_*</code> , <code>APP_BASE_URL</code>	Server	Same as Sections 5-6	via <code>sendPushFor-Transition.js</code>

4. Files involved, in order

File	Purpose
<code>pages/api/cron/mark-overdue.js</code>	The scheduled endpoint

Essential auth-gate — **fails closed** if the secret isn't configured at all:

```
function isAuthorized(req) {
  const secret = process.env.CRON_SECRET;
  if (!secret) return false;
  if (req.headers.authorization === `Bearer ${secret}`) return true;
  if (req.query?.secret === secret) return true;
  return false;
}
```

Query shape that avoids needing a composite index (two equality/in filters, no orderBy):

```
const snapshot = await db.collection("reports")
  .where("status", "in", ACTIVE_STATUSES)
  .where("overdue", "=", false)
  .get();
// due-date comparison done client-side in JS, not as a Firestore range filter
```

5. Order of operations / dependency notes - Write the overdue: true batch update **before** firing notifications (`Promise.allSettled`), and never let a notification failure roll back or block the write — the write is the source of truth; notification is best-effort. - Because this endpoint mirrors client-side logic that also exists in `src/utils.js` (`markOverdueReports`, called on app load as a redundant catch-up mechanism), keep the active-status list in both

places in sync manually — there is no shared import between the Admin SDK server context and the client bundle in this project's structure.

6. Common pitfalls encountered in this project - Forgetting the `!secret` fail-closed check means an unset `CRON_SECRET` env var (e.g. a fresh deploy before you've configured it) makes the endpoint **unauthenticated by omission** rather than simply broken — always fail closed on missing config for anything auth-related. - Firestore `where(..., "in", [...])` combined with a second `where(..., "=", ...)` needs no composite index (both are satisfiable by automatic single-field indexes merged by the query planner) — but adding an `orderBy` on top of this shape *does* require one; keeping the due-date comparison as plain JS filtering (rather than a third `where` range clause) was a deliberate choice to keep this query index-free. - Two independent code paths marking things overdue (this cron job, and the client-side `markOverdueReports` on app load) must both scope their query to `overdue == false` so a report is never double-notified — verify this invariant explicitly if you add a third path.

7. Minimal reproduction checklist - [] Generate and store `CRON_SECRET` server-side only. - [] Implement `isAuthorized` to fail closed when the secret is unset. - [] Accept both a Bearer header and a query-param fallback for scheduler compatibility. - [] Query with `in + ==` only (no `orderBy`) to avoid a composite index requirement; do date comparisons in JS. - [] Batch the write, then fire notifications via `Promise.allSettled` without blocking on their result. - [] Register the endpoint with an external scheduler (cron-job.org or Vercel Cron) with the secret attached. - [] Confirm the route isn't swallowed by an SPA catch-all rewrite.

8. Image upload/compression pipeline

1. What it does Client-side canvas-based image resize + JPEG re-encode before storing as a base64 string in a Firestore doc, with the image payload deliberately split into a **separate** document from its parent record so list-view queries never download image bytes.

2. Outside-the-codebase setup None — pure client-side canvas API, no external service. (No Firebase Storage bucket is used for this — images are stored as base64 strings directly in Firestore documents, which is a deliberate trade-off; see pitfalls.)

3. Environment variables required None.

4. Files involved, in order

File	Purpose
Any form component with a file input	Guards raw file size before reading
Compression helper (inline or shared)	Canvas resize + re-encode
<code>firestore.rules</code> (match <code>/reportImages/{reportId}</code>)	Field-shape + size validation on write, and read scoping

Essential compression function:

```
const compressImage = (file) => new Promise((resolve, reject) => {
  const reader = new FileReader();
  reader.onerror = () => reject(new Error("Could not read the image file."));
  reader.readAsDataURL(file);
  reader.onload = (event) => {
    const img = new Image();
    img.onerror = () => reject(new Error("Could not process the image."));
    img.src = event.target.result;
    img.onload = () => {
```

```

    const canvas = document.createElement("canvas");
    const maxWidth = 600;
    let { width, height } = img;
    if (width > maxWidth) { height = (height * maxWidth) / width; width = maxWidth; }
    canvas.width = width; canvas.height = height;
    canvas.getContext("2d").drawImage(img, 0, 0, width, height);
    resolve(canvas.toDataURL("image/jpeg", 0.5)); // quality 0.5
  };
};
});

```

Pre-read size guard (before the expensive `FileReader.readAsDataURL` call):

```

const MAX_IMAGE_BYTES = 15 * 1024 * 1024;
if (file.size > MAX_IMAGE_BYTES) { /* reject before reading into memory */ }

```

Firestore rule enforcing the split-document pattern and payload shape:

```

match /reportImages/{reportId} {
  allow create: if isVerified() && isStaff() &&
    reportData().reporterId == getUserID() &&
    reportData().status == 'incoming' &&
    request.resource.data.keys().hasOnly(['image']) &&
    request.resource.data.image is string &&
    request.resource.data.image.size() > 0;
}

```

5. Order of operations / dependency notes

1. Guard raw file size **before** calling `FileReader.readAsDataURL` — the whole point is avoiding holding an uncompressed multi-MB base64 string in memory.
2. Revoke any `URL.createObjectURL` preview blob (`URL.revokeObjectURL`) whenever the preview changes or the component unmounts — do this in a `useEffect` cleanup keyed on the preview state, not just on submit.
3. Write the parent record first, then the separate image document, keyed by the same ID (e.g. `reportImages/{reportId}` where `reportId` is the just-created parent doc's ID) — if the image write fails, don't roll back the parent record if the image was optional; instead surface a distinct "saved, but photo failed" message.
4. Fetch the image document lazily — only when a detail view actually opens (a single `getDoc`), never as part of the list-view query.

6. Common pitfalls encountered in this project

- Firestore documents have a 1MB size limit — base64-encoding inflates binary size by ~33%, so the compression step's `maxWidth/quality` settings must be tuned conservatively (this project uses 600px width, quality 0.5 for reports; 800px/0.6 for completion photos) or large-but-valid photos will fail the write with no client-side warning until the actual `setDoc/addDoc` call throws.
- An `img.onerror` handler is required — without it, a corrupt or unsupported file leaves the compression Promise pending forever, and a submit button spins indefinitely with no error surfaced.
- Storing images directly on the parent document (instead of a separate sibling doc) means every list-view snapshot listener re-downloads every image payload on every load — moving images to a separate, lazily-fetched document was a direct fix for unbounded list-view read cost, not a stylistic choice.
- Security rules for the split image collection typically need to `get()` the parent document to check ownership/status (`reportData()` helper) — this adds a read per rule evaluation and requires the parent doc to still exist at delete time, so image-cleanup must happen **before** parent-record deletion, not after.

7. Minimal reproduction checklist

- [] Guard raw file size before `FileReader.readAsDataURL`.
- [] Canvas-resize to a conservative max dimension; re-encode as JPEG at a conservative quality (start ~0.5–0.6).
- [] Add an `img.onerror` handler so unsupported files reject instead of hanging.
- [] Store the compressed base64 string in a **separate** Firestore document from

the parent record, keyed by the parent's ID. - [] Fetch that document lazily (detail view only), never in a list-view query/listener. - [] Revoke object-URL previews on change/unmount. - [] Write Firestore rules for the image doc that validate `keys().hasOnly([...])` and non-empty string size, scoped by a `get()` on the parent doc.

9. PDF generation (jsPDF)

1. What it does Client-side PDF generation with jsPDF for two distinct documents: a single-record report (materials/procurement form with tables and signature lines) and a full dashboard analytics report (KPI cards, bar charts, progress bars, multi-page layout) — both fully drawn with the low-level `doc.text/doc.rect/doc.line` primitives, no HTML-to-PDF conversion.

2. Outside-the-codebase setup None — jsPDF runs entirely client-side, no external service or API key.

3. Environment variables required None.

4. Files involved, in order

File	Purpose
<code>src/utils.js</code> (<code>generatePDFReport</code> , <code>generateDashboardStatsPDF</code>)	Both PDF builders
A static logo asset imported and passed to <code>doc.addImage</code>	Header branding

Essential init + save pattern:

```
import { jsPDF } from "jspdf";
const doc = new jsPDF({ unit: "pt", format: "a4" });
// ... draw with doc.text(), doc.rect(), doc.line(), doc.addImage(), doc.splitTextToSize()
doc.save(`Report_${id}.pdf`);
```

Multi-page pattern used for the dashboard report — track a manual y cursor and page-break explicitly:

```
const ensureSpace = (height) => {
  if (y + height > pageHeight - 55) { doc.addPage(); addPageHeader(); }
};
```

5. Order of operations / dependency notes - `doc.addImage`(`logoSrc`, "PNG", `x`, `y`, `w`, `h`) needs the image already resolvable synchronously as a bundler-imported asset path (not a remote URL fetched at call time) — wrap in `try/catch` since a bad path silently fails to render rather than throwing in some environments. - Any multi-page document needs its own manual pagination bookkeeping (`y` cursor + `ensureSpace` guard before each section) since jsPDF does not auto-flow content across pages. - Footer/page-numbering (Page X of Y) can only be correctly written **after** all content is drawn, since the final page count isn't known until then — iterate `doc.internal.getNumberOfPages()` in a final pass over every page via `doc.setPage(n)`.

6. Common pitfalls encountered in this project - jsPDF's default Helvetica font does not reliably render some Unicode currency symbols (this project's ₵ Ghana cedi sign) — the fix used here is formatting amounts as "GHS 1,234" (ASCII) instead of embedding the symbol, rather than trying to load a custom font just for one character. - `doc.splitTextToSize`(`text`, `maxWidth`) must be used before manually breaking any variable-length text into a table cell or paragraph — writing unwrapped text past a column's width silently overflows into adjacent

columns with no error. - Table rows drawn manually (rect + text per cell, no plugin) require you to track row height and cumulative y yourself; a jspdf-autotable plugin would remove this boilerplate but wasn't used here — worth evaluating for a new project if table-heavy PDFs are common.

7. Minimal reproduction checklist - [] npm install jspdf. - [] Build via a manual y cursor + a set of small draw-primitive helper functions (addLine, addSectionGap, drawRoundedCard, etc.) rather than one giant function. - [] Use doc.splitTextToSize before drawing any variable-length text block. - [] For multi-page docs, write an ensureSpace(height) guard called before every section, and a final pass to stamp page numbers after all content is drawn. - [] Avoid non-ASCII currency/special symbols with the default font; substitute an ASCII abbreviation or embed a custom font if the symbol is required. - [] Wrap doc.addImage in try/catch for logo/branding assets.

10. Bounded/paginated Firestore queries + composite indexes

1. What it does Replaces unbounded, always-live onSnapshot listeners on entire collections with either (a) server-side cursor pagination ordered by a single guaranteed-present date field, or (b) a bounded one-time fetch + client-side substring search — plus a separate lightweight getCountFromServer aggregation for accurate totals, avoiding the cost/latency growth of “read everything on every write, forever.”

2. Outside-the-codebase setup 1. The first time a genuinely new query “shape” (a specific combination of equality/in filters + orderBy) runs against Firestore, if it needs a composite index the SDK throws an error containing a **direct console link** to auto-create it — click that link once per shape. 2. Alternatively, predeclare indexes in firestore.indexes.json and deploy: firebase deploy --only firestore:indexes. 3. Composite indexes are free on the Spark (free) plan — no billing tier requirement specifically for this, unlike Cloud Functions/scheduled jobs.

3. Environment variables required None beyond standard Firebase client config (Section 1).

4. Files involved, in order

File	Purpose
firestore.indexes.json	Predeclared composite indexes (optional if you rely on the auto-create-link flow)
Any list-view component	Cursor-pagination hook

Essential cursor-pagination shape:

```
const cursorCacheRef = useRef({ 1: null });
const q = query(
  collection(db, "reports"),
  ...baseEqualityFilters,
  orderBy("dateSent", "desc"), // the ONE field guaranteed present on every doc
  ...(cursor ? [startAfter(cursor)] : []),
  limit(pageSize),
);
onSnapshot(q, (snapshot) => {
  cursorCacheRef.current[page + 1] = snapshot.docs[snapshot.docs.length - 1];
});
```

Separate, cheap total-count query (no orderBy, so no extra composite index needed beyond the base equality filters):

```
const countSnap = await getCountFromServer(query(collection(db, "reports"), ...baseEqualityFilters));
```

Bounded fallback for free-text search (Firestore has no native substring search):

```
const snap = await getDocs(query(collection(db, "reports"), ...baseFilters, orderBy("dateSent")));
const results = snap.docs.map(...).filter(matchesSearchLocally);
```

5. Order of operations / dependency notes - Reset the pagination cursor cache ({ 1: null }) whenever the query “shape” changes — any equality filter, role/user scoping, or time-range filter — not just when the page number changes. - The per-page-display sort (e.g. by a status-specific date field like dateAccepted) must be applied **client-side, after** fetching a page, using a fallback chain (dateAccepted → dateSent → epoch) — never pass that field to Firestore’s own orderBy, see pitfall below. - A getCountFromServer call is a separate, cheaper request (billed as one aggregation read, not a per-document read) — don’t derive total count from snapshot.docs.length on a paginated query, since that only reflects the current page.

6. Common pitfalls encountered in this project - orderBy(field) silently drops any document that doesn’t have that field set at all — this is the single highest-value pitfall in this section. Ordering a paginated query by a status-specific date field (e.g. dateClosed) means any older document written before that field existed, or that some code path never set, simply vanishes from paginated results — while an equality-only count query (no orderBy) still counts it. This produces the confusing symptom of “pagination says there are N reports, but fewer cards render than N.” Fix: always drive the server-side orderBy/cursor off one field guaranteed present on every document (this project standardized on dateSent, set at creation time on every record), and do any secondary/display-only sorting by a different field entirely client-side after fetching. - Switching a collection from one always-live onSnapshot on the whole thing to several small, purpose-scoped bounded queries (e.g. one live listener per status for badge counts, capped at a few hundred docs each) trades a small amount of live-ness for a hard ceiling on read cost — decide deliberately per use case whether true real-time matters (e.g. “did a report I’m viewing right now change status” — yes, keep live) vs. whether an explicit refresh button is acceptable (e.g. an admin analytics dashboard — this project switched that one to explicit-refresh-only). - Debounce free-text search input (e.g. 350ms) before it drives a new bounded fetch — searching on every keystroke multiplies read cost for no benefit.

7. Minimal reproduction checklist - [] Ensure every document in a paginated collection always has one non-optional field for ordering (set it at creation time, never conditionally). - [] Drive orderBy + startAfter cursors off that one guaranteed field only. - [] Do any status/context-specific display sorting client-side, after fetching, with an explicit fallback chain. - [] Use getCountFromServer for totals — never snapshot.docs.length on a limited query. - [] Cache pagination cursors keyed by page number; reset the cache whenever the query’s filter “shape” changes. - [] For free-text search Firestore can’t do server-side, fetch a bounded, generously-sized window and filter client-side; debounce the input driving it. - [] Click through the auto-generated composite-index console links the first time each new query shape runs, or predeclare them in firestore.indexes.json. - [] Deliberately choose live (onSnapshot) vs. bounded-explicit-refresh per view based on whether millisecond freshness actually matters there.

11. API routing on Vercel (SPA + serverless functions coexisting)

(Folded as a pitfall reference — see Section 4, “Order of operations,” point 6, for the full explanation. Summarized here for quick lookup.)

Essential vercel.json rewrite — must exclude `/api/` and any real static file (matched by extension) from the SPA catch-all, or API calls resolve to `index.html` instead of your function:

```
{
  "rewrites": [
    { "source": "/((?!api/|.*\\.([a-zA-Z0-9]+)$).*)", "destination": "/index.html" }
  ]
}
```

Local dev: plain vite dev does not proxy `/api/*` to Vercel's serverless runtime — use `vercel dev`, or add a `server.proxy["/api"]` entry in `vite.config.js` pointing at a separately-running vercel dev process.

12. Median native app wrapper (Android/iOS shell + native OneSignal bridge)

(Depends on: Section 5 for the OneSignal side of this integration)

1. What it does Wraps the deployed web app as a native Android/iOS app via Median (formerly GoNative), including native push notification delivery through Median's own OneSignal plugin (bridged to the same OneSignal App ID used for web), splash screens, status bar theming, and link-handling rules (which URLs open in-app vs. an external browser/app-browser view).

2. Outside-the-codebase setup 1. Create a Median account → **New App** → point `initialUrl` at your deployed web app's URL. 2. Median dashboard → App Icon & Splash Screen → upload icons/splash assets per platform (iOS/Android, light/dark) — these get baked into the native build, not read from your web repo. 3. Median dashboard → Native Plugins → **OneSignal** → enable it → paste the **same OneSignal App ID** used for Web Push (Section 5) so native and web subscribers land in the same OneSignal app. 4. Median dashboard → Native Plugins → OneSignal → configure the custom notification sound (upload `.caf` for iOS / equivalent for Android) and create an Android notification channel — copy the resulting sound filename and channel UUID into your server-side send code (`sendPushForTransition.js`, `IOS_SOUND` / `ANDROID_CHANNEL_ID` constants). 5. Median dashboard → URL/Navigation rules (`regexInternalExternal`) → define which URL patterns open in-app (`internal`), in a native browser overlay (`appbrowser`), or a full external browser (`external`) — e.g. keep your own domain `internal`, open social/map links `appbrowser`, and treat anything unmatched as `external`. 6. Median dashboard → Build → generate the Android `.aab/APK` and/or iOS build. iOS distribution requires an active Apple Developer Program membership; Android requires a Google Play Console account for store distribution (side-loading the APK does not). 7. Median injects `window.median` into the webview automatically (`injectMedianJS: true` in the exported config) — no separate script tag needed in your own `index.html` for this.

3. Environment variables required None — Median configuration lives entirely in Median's own dashboard/exported JSON config, not in your app's `.env`. The OneSignal App ID is duplicated by value (not by env var) into both your `index.html`'s Web SDK init (Section 5) and Median's dashboard field.

4. Files involved, in order

File	Purpose
Median's exported config JSON (downloaded from dashboard, kept for reference/version control — not read by your build)	Documents the full native config: icons, splash, nav rules, OneSignal plugin settings

File	Purpose
src/lib/push.js	Detects <code>window.median?.onesignal</code> and branches to the native login call (see Section 5)

The only code-side integration point is the `window.median` detection branch already covered in Section 5 — Median itself requires no SDK install in your web repo.

5. Order of operations / dependency notes 1. Get the web app fully working (Auth, push via Web SDK, etc.) **before** wrapping with Median — debugging is dramatically easier in a browser than in a native shell. 2. Set up the OneSignal Web Push app (Section 5) first; Median’s native OneSignal plugin is configured to point at that *same* App ID, not a separate one — creating a second OneSignal app for native would split your subscriber base. 3. Median rebuilds are required (not just a redeploy of your web app) whenever you change anything in Median’s own dashboard config (icons, splash, nav rules, plugin settings) — your web app’s own deploys (Vercel, etc.) go live instantly inside the existing native shell without a rebuild, since the shell just loads `initialUrl`, but dashboard-level native config changes do not take effect until you trigger a new native build.

6. Common pitfalls encountered in this project - `window.median?.onesignal` is not synchronously available at page load inside the native webview — must be polled with a timeout rather than checked once (see Section 5’s `waitForMedianOneSignal`). - The custom notification sound and Android channel ID are configured **only** in Median’s dashboard, but must also be hardcoded into your server-side send payload (`ios_sound`, `android_channel_id` fields) — these two configuration surfaces (Median dashboard vs. your server code) can silently drift out of sync if the sound/channel is ever changed in one place and not the other, with no build-time error to catch the mismatch. - URL navigation rules (`regexInternalExternal`) are evaluated in order, and the last matching catch-all rule (`.* → external`) means any link pattern you forget to explicitly mark `internal` opens in a full external browser instead of staying in-app — review this list whenever adding a new external integration (e.g. a payment provider redirect) that should stay in-app. - Native app review (both Apple and Google) can reject a pure webview wrapper for insufficient native functionality — having genuine native integrations (push notifications, pull-to-refresh, proper offline handling — all configurable in Median’s dashboard) materially helps pass review, not just cosmetic.

7. Minimal reproduction checklist - [] Get the web app (Auth, Section 5’s Web Push) fully working standalone first. - [] Create a Median app pointed at the deployed web URL. - [] Enable Median’s OneSignal native plugin using the **same** OneSignal App ID as your Web Push setup. - [] Configure custom sound/notification channel in Median’s dashboard, then copy the exact filename/UUID into your server-side send code. - [] Define explicit `internal/appbrowser/external` URL rules for every link pattern your app can produce; don’t rely on the catch-all. - [] Add the `window.median?.onesignal` detection + polling branch to your push-linking code. - [] Trigger a fresh Median build after any dashboard-level config change — web deploys alone do not propagate these. - [] Confirm Apple Developer / Google Play accounts are set up before attempting store distribution builds.